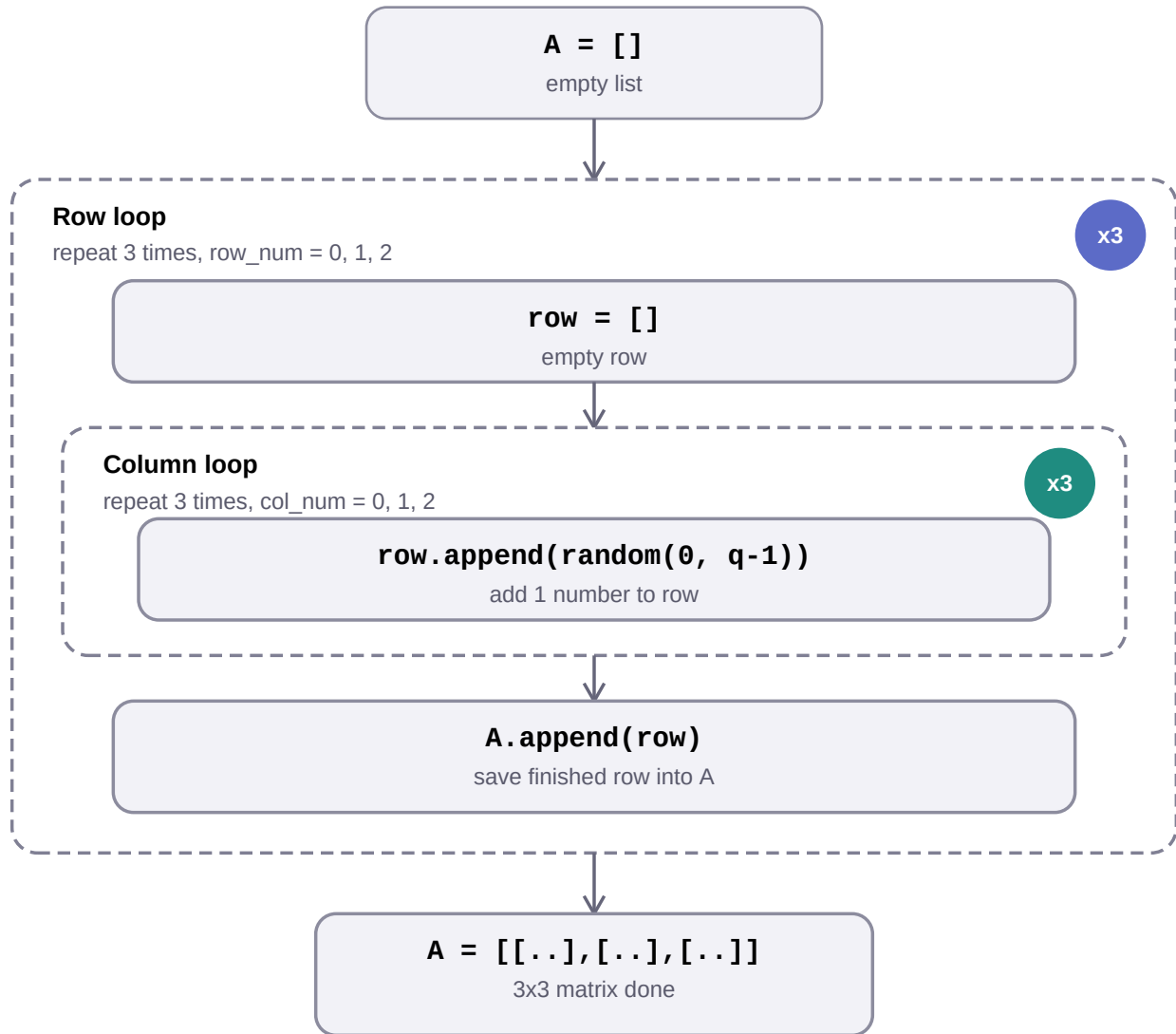


# Toy Integer Kyber (3x3)

Code walkthrough: how A, S, e and b are built

This document accompanies `toy_integer_kyber_3x3.py`.  
It breaks down each block of Bob's key generation step  
(A, S, e, and the calculation of  $b = A \times S + e \bmod q$ )  
into a simple diagram and a plain-English description,  
line by line, matching the code in the file exactly.

# Building A - the 3x3 matrix



A starts as an empty list.

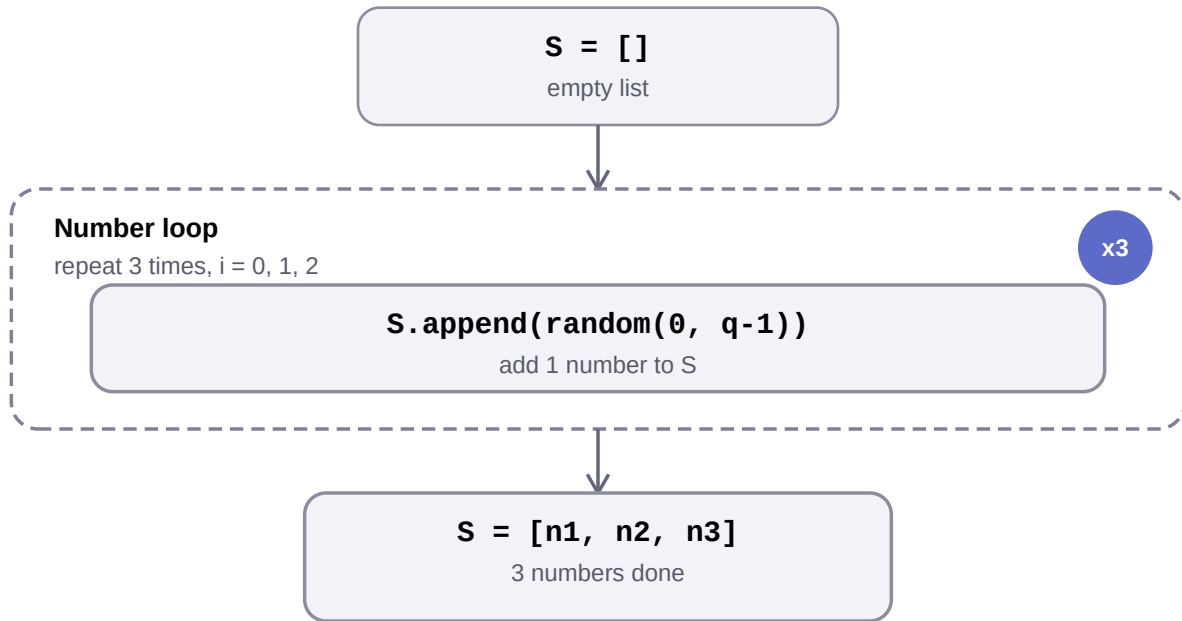
The row loop repeats 3 times (once per row).

Each repeat: start a fresh empty row, then the column loop repeats 3 times, adding one random number (0 to q-1) each time.

Once the row has 3 numbers, it is saved onto the end of A.

After 3 repeats of the row loop, A holds 3 rows of 3 numbers - a 3x3 matrix.

## Building S - Bob's secret vector



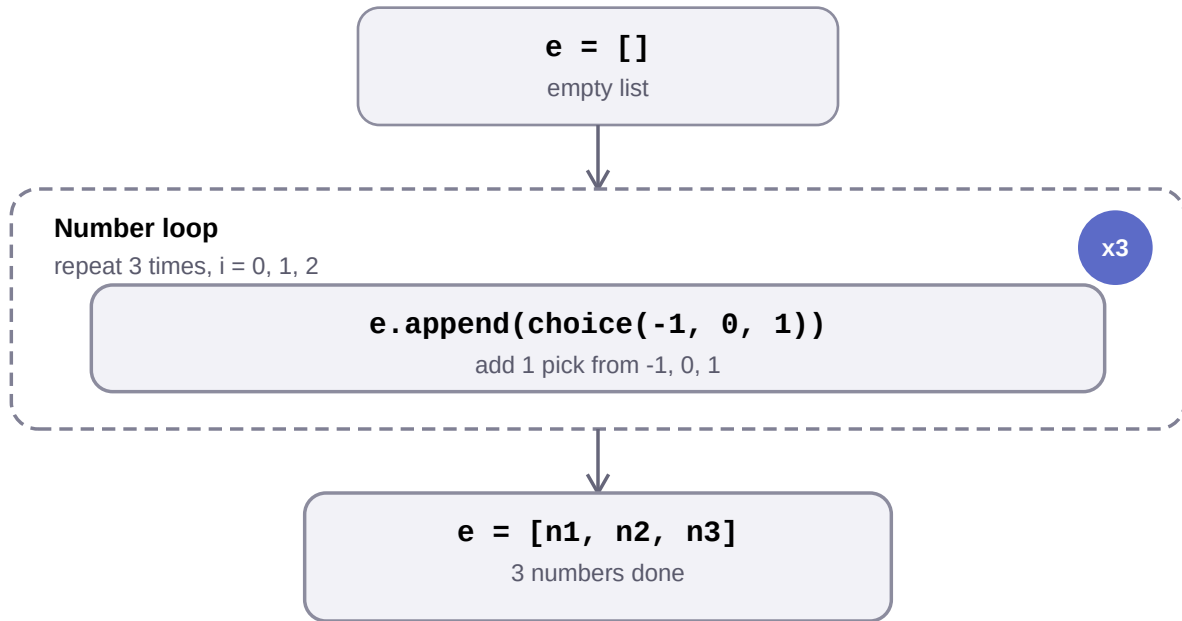
$S$  starts as an empty list.

The loop repeats 3 times, once per entry.

Each repeat, one random number (0 to  $q-1$ ) is added to  $S$ .

No inner loop is needed here -  $S$  is a single flat list (a vector), not a grid of lists like  $A$ .

## Building e - the small error vector

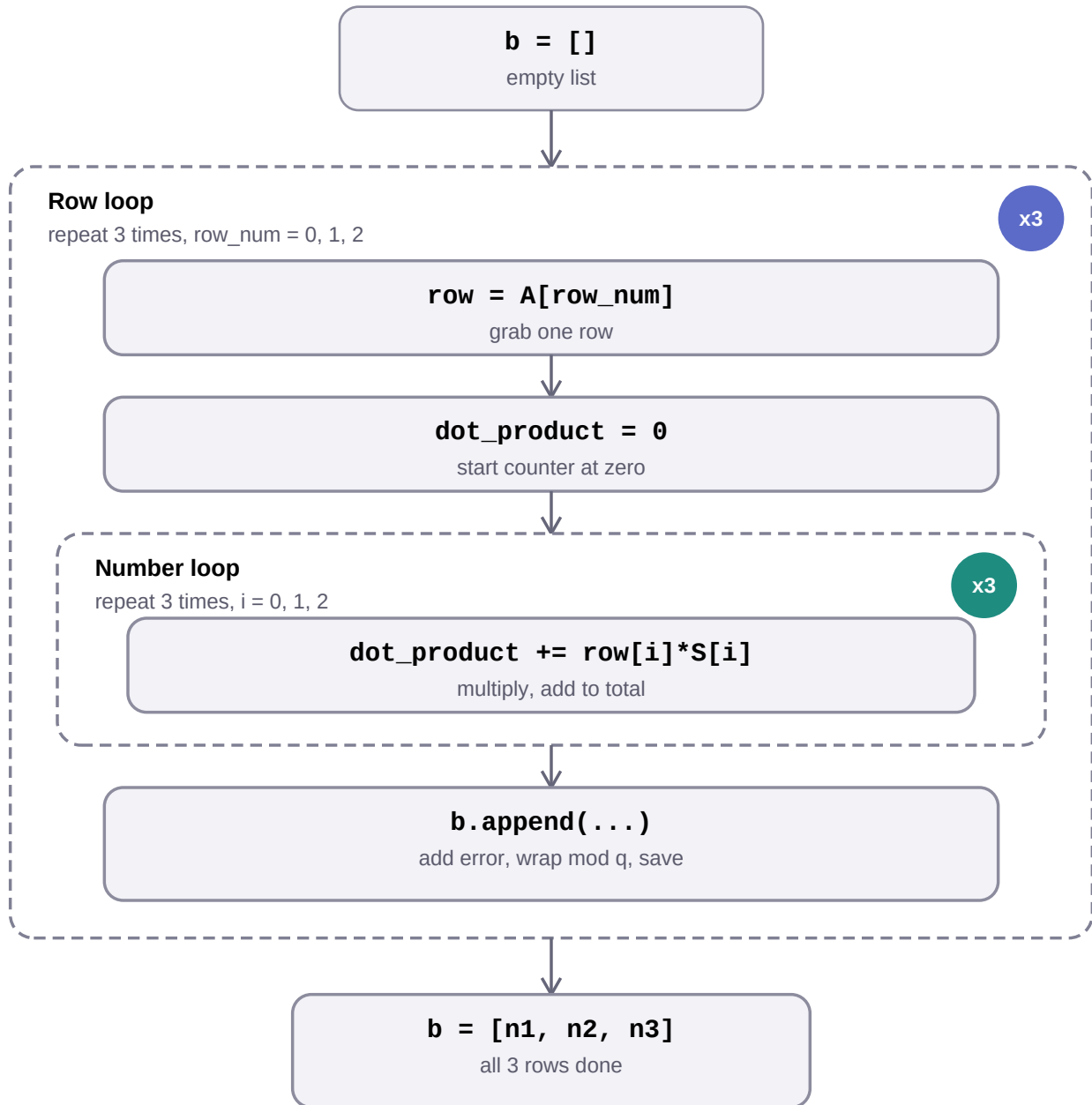


**e** starts as an empty list, same shape as **S**.

The one difference: `random.choice([-1, 0, 1])` only ever picks from those 3 exact values, unlike `random.randint` which can pick any number in a range.

This keeps the error small, matching the LWE requirement that **e** must be a small, secret nudge.

## Calculating $b = A \times S + e \pmod q$



$b$  starts as an empty list.

The row loop repeats 3 times, once per row of  $A$ .

For each row: grab that row, start a counter at 0, then the number loop multiplies matching entries of row and  $S$  and adds them into the counter (the dot product).

Once that finishes, this row's error is added, the result is wrapped mod  $q$ , and saved onto  $b$ . Repeat for all 3 rows.